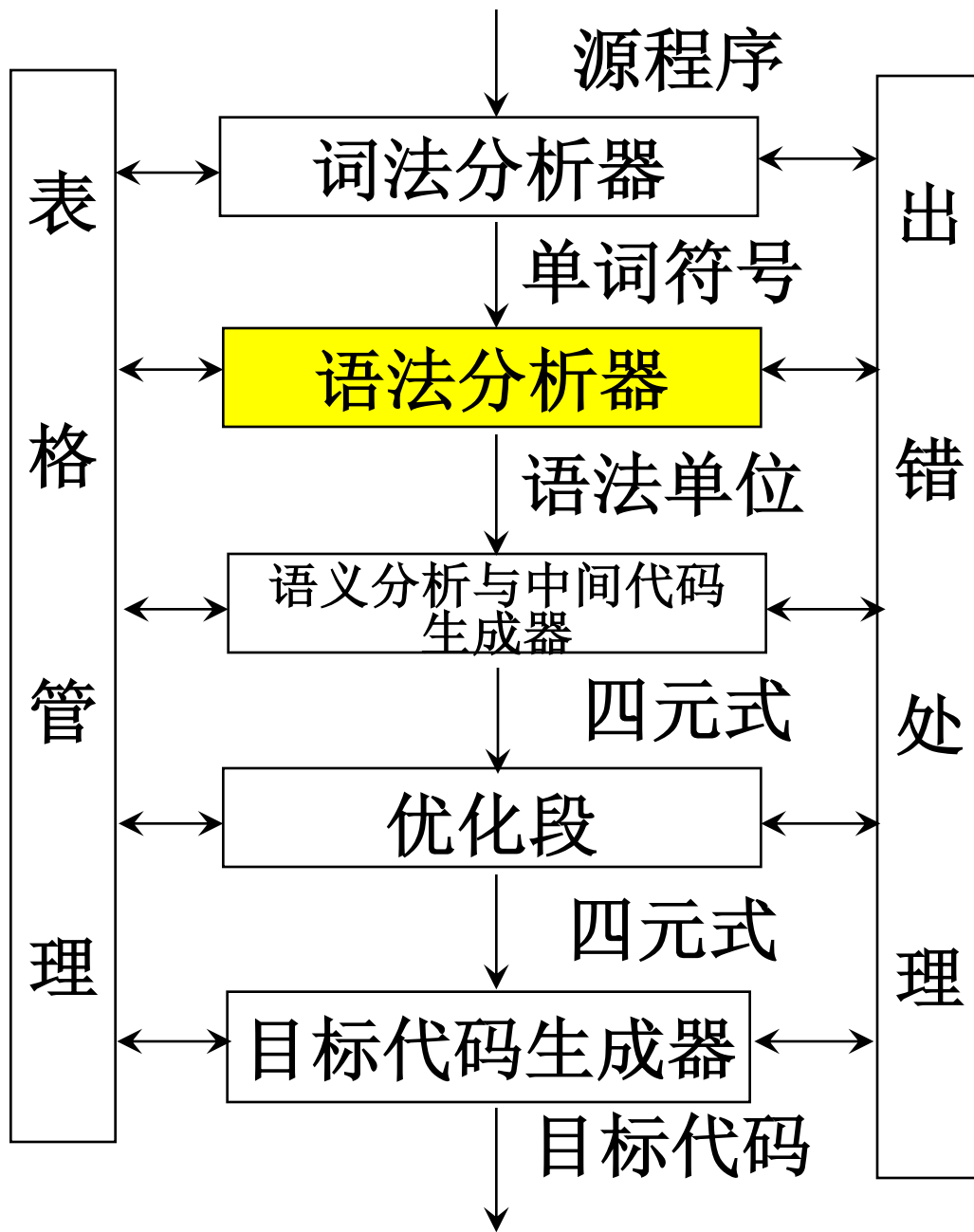


第四章

语法分析—自上而下分析



第四章 语法分析-自上而下分析

- 4.1 语法分析器的功能
- 4.2 自上而下分析面临的问题
- 4.3 LL(1)分析
- 4.4 递归下降分析程序构造
- 4.5 预测分析程序
- 4.6 LL(1) 分析中的错误处理

4.1 语法分析器的功能

- 语法分析：分析并判定程序的语法结构**是否符合语法规则**。
- 语法规则的描述：**上下文无关文法**
- 语法分析器的工作：**按文法的产生式，识别输入符号串是否为一个句子**。
 - 输入串：单词符号组成的有限序列。
 - 建立一棵与输入串相匹配的语法分析树。

- 按照语法分析树的建立方法，把语法分析方法分成两类：
 - 自上而下分析法：从文法的开始符号出发，向下推导，推出句子（输入串）。
 - 自下而上分析法：从输入串开始，逐步进行归约，直至归约到文法的开始符号。
 - 归约，是指根据文法的产生式，把产生式的右部替换成左部符号。

4.2 自上而下分析面临的问题

- 自上而下分析的主旨：
 - 对任何输入串，试图用一切可能的办法，从文法开始符号(根结)出发，自上而下地为输入串建立一棵**语法树**。或者说，为输入串寻找一个最左推导。

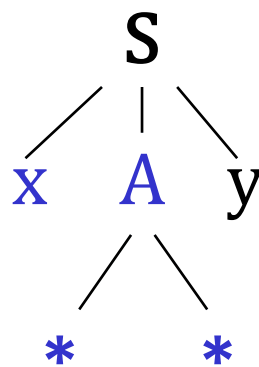
- 例3.4.1 假定有文法 $G(S)$:

(1) $S \rightarrow xAy$

(2) $A \rightarrow **|*$

分析输入串 $x*y$ (记为 α)。

$x * y$
↑
IP



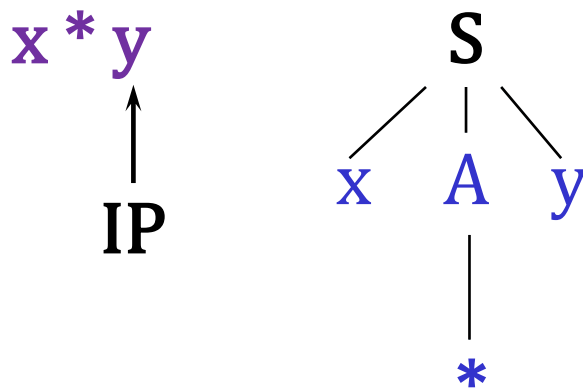
失败!

- 例3.4.1 假定有文法 $G(S)$:

(1) $S \rightarrow xAy$

(2) $A \rightarrow **|*$

分析输入串 $x*y$ (记为 α)。



α 是一个句子。

- 困难:
 - 文法的左递归问题
 - 一个文法是含有左递归的，如果存在非终结符 P : $P \Rightarrow^* P\alpha$
 - 含有左递归的文法将使自上而下的分析陷入无限循环。
 - 回溯

4.3 LL(1)分析法

- 构造不带回溯的自上而下分析算法
 - 消除文法的左递归
 - 克服回溯

4.3.1 左递归的消除

- 直接消除产生式中的左递归

假定关于非终结符P的规则为

$$P \rightarrow P\alpha \mid \beta$$

其中 β 不以P开头。

从P推出的符号串： β 开头后跟0个或多个 α 。

改写P的规则：

$$P \rightarrow \beta P'$$

$$P' \rightarrow \alpha P' \mid \varepsilon$$

- 例4.2 文法G(E):

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid i$$

消除直接左递归:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid i$$

(4.2)

- 假定P的全部产生式:

$$P \rightarrow P\alpha_1 \mid P\alpha_2 \mid \dots \mid P\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

其中，每个 α 都不等于 ε ，每个 β 都不以P开头

消除P的直接左递归性:

$$P \rightarrow \beta_1 P' \mid \beta_2 P' \mid \dots \mid \beta_n P'$$

$$P' \rightarrow \alpha_1 P' \mid \alpha_2 P' \mid \dots \mid \alpha_m P' \mid \varepsilon$$

- 例如文法G(S):

$S \rightarrow Qc|c$

$Q \rightarrow Rb|b$

$R \rightarrow Sa|a$ (4.3)

不具有直接左递归，但S、Q、R都是左递归的

$S \Rightarrow Qc \Rightarrow Rbc \Rightarrow Sabc$

$Q \Rightarrow Rb \Rightarrow Sab \Rightarrow Qcab$

$R \Rightarrow Sa \Rightarrow Qca \Rightarrow Rbca$

- 消除左递归算法:

1. 把文法G的所有非终结符按任一种顺序排列成 $P_1, P_2, \dots, P_n$; 按此顺序执行;

2. **FOR** $i:=1$ **TO** n **DO**

BEGIN

FOR $j:=1$ **TO** $i-1$ **DO**

把形如 $P_i \rightarrow P_j \gamma$ 的规则改写成 $P_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$ 。

其中 $P_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ 是关于 P_j 的所有规则;

消除关于 P_i 规则的直接左递归性

END

3. 化简由2所得的文法。去除那些从开始符号出发永远无法到达的非终结符的产生规则。

- 例4.3 消除文法的左递归。

$G(S):$

$S \rightarrow Qc | c$

$Q \rightarrow Rb | b$

$R \rightarrow Sa | a$

$G(S):$

$R \rightarrow Sa|a$

$Q \rightarrow Rb|b$

$S \rightarrow Qc|c$

非终结符顺序:

P_1	P_2	P_3
R	Q	S

$i=1:$ $R \rightarrow Sa|a$
 无直接左递归

$G(S):$

$R \rightarrow Sa|a$

$Q \rightarrow Rb|b$

$S \rightarrow Qc|c$

非终结符顺序:

P_1	P_2	P_3
R	Q	S

$i=2:$ $Q \rightarrow Rb|b$ 改写为:

$Q \rightarrow Sab|ab|b$

无直接左递归

$G(S)$:

$R \rightarrow Sa|a$

$Q \rightarrow Sab|ab|b$

$S \rightarrow Qc|c$

非终结符顺序:

P_1	P_2	P_3
R	Q	S

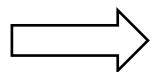
$i=3$: $S \rightarrow Qc|c$ 改写为:

$S \rightarrow Sabc|abc|bc|c$

消除直接左递归

$S \rightarrow abcS'|bcS'|cS'$

$S' \rightarrow abcS'|\epsilon$



$G(S)$:

$R \rightarrow Sa|a$

$Q \rightarrow Sab|ab|b$

$S \rightarrow abcS'|bcS'|cS'$

$S' \rightarrow abcS'|\epsilon$



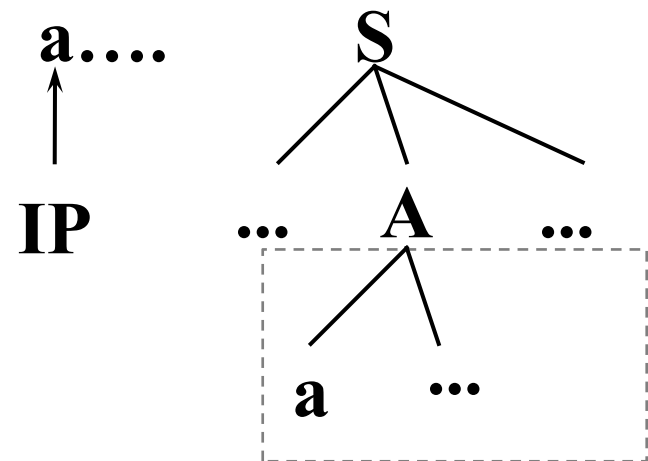
$S \rightarrow abcS'|bcS'|cS'$

$S' \rightarrow abcS'|\epsilon$

(4.4)

4.3.2 消除回溯、提左因子

- 消除回溯须保证：对文法的任何非终结符，当要它去匹配输入串时，能够根据它所面临的输入符号准确地指派它的一个候选去执行任务，并且此候选的工作结果应是确信无疑的。
- $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$



- 令 G 是一个不含左递归的文法，
 α 为 G 的任意非终结符的任意候选，
 α 的**终结首符集** $\text{FIRST}(\alpha)$ 为：

$$\text{FIRST}(\alpha) = \{a \mid \alpha \xRightarrow{*} a \dots, a \in V_T\}$$

若 $\alpha \xRightarrow{*} \varepsilon$ ，则 $\varepsilon \in \text{FIRST}(\alpha)$ 。

- 如果非终结符 A 的所有候选的首符集两两不相交，即 A 的任何两个不同候选 α_i 和 α_j

$$\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \phi$$

当要求 A 匹配输入串时， A 就能根据它所面临的第一个输入符号 a ，准确地指派某一个候选前去执行任务。
 这个候选就是那个终结首符集含 a 的 α 。

构造文法符号的FIRST集(p78)

对每一文法符号 $X \in V_T \cup V_N$ 构造FIRST(X)

方法:

连续使用下面的规则，直至每个集合FIRST不再增大为止。

(1) 若 $X \in V_T$, 则 $\text{FIRST}(X) = \{X\}$ 。

(2) 若 $X \in V_N$,

- 有产生式 $X \rightarrow a...$, 把 a 加入到 $\text{FIRST}(X)$ 中;
- 若 $X \rightarrow \varepsilon$ 是产生式, 则把 ε 加到 $\text{FIRST}(X)$ 中。

(3) 若 $X \in V_N$, 且有产生式 $X \rightarrow Y_1 Y_2 \dots Y_k$, 其中 $k \geq 1$:

- $\text{FIRST}(Y_1)$ 中的非 ε -元素都加到 $\text{FIRST}(X)$ 中;
- 设 $1 \leq j \leq i-1$, $\text{FIRST}(Y_j)$ 都含有 ε , $\text{FIRST}(Y_i)$ 中的非 ε -元素加到 $\text{FIRST}(X)$ 中;
- 若所有的 $\text{FIRST}(Y_j)$ 均含有 ε , $1 \leq j \leq k$, 把 ε 加到 $\text{FIRST}(X)$ 中。

例4.7

$G(E)$:

$E \rightarrow TE'$

$E' \rightarrow +T E' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$

$\text{FIRST}(E) = \text{FIRST}\{T\} \setminus \{\varepsilon\} = \{ (, i \}$

$\text{FIRST}(E') = \{ +, \varepsilon \}$

$\text{FIRST}(T) = \text{FIRST}(F) \setminus \{\varepsilon\} = \{ (, i \}$

$\text{FIRST}(T') = \{ *, \varepsilon \}$

$\text{FIRST}(F) = \{ (, i \}$

4.3.2 消除回溯、提左因子

- 如果非终结符A的所有候选的首符集两两不相交，即A的任何两个不同候选 α_i 和 α_j

$$\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \phi$$

当要求A匹配输入串时，A就能根据它所面临的第一个输入符号a，准确地指派某一个候选前去执行任务。

- 许多文法它的所有候选的首符集并非两两不相交。

stmt $\rightarrow$ if expr then stmt else stmt
| if expr then stmt
| other

- **提取公共左因子**，把一个文法改造成任何非终结符的所有候选首符集两两不相交。

假定关于A的规则是

$$A \rightarrow \delta\beta_1 \mid \delta\beta_2 \mid \dots \mid \delta\beta_n \mid \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m$$

(其中，每个 γ 不以 δ 开头)

改写规则：

$$A \rightarrow \delta A' \mid \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

- 经过反复提取左因子，就能够把每个非终结符(包括新引进者)的所有候选首符集变成为两两不相交。

- 提取公共左因子

$\text{stmt} \rightarrow \text{if expr then stmt else stmt}$
 $\quad | \text{if expr then stmt}$
 $\quad | \text{other}$



$\text{stmt} \rightarrow \text{if expr then stmt optional_else_part}$
 $\quad | \text{other}$
 $\text{optional_else_part} \rightarrow \text{else stmt} \mid \varepsilon$

4.3.3 LL(1)分析条件

- 当一个文法不含左递归，并且满足每个非终结符的所有候选首符集两两不相交的条件，是不是就一定能进行有效的自上而下分析了呢？
- 如果空字 ϵ 属于某个非终结符的候选首符集，问题就比较复杂。

4.3.3 LL(1)分析条件

- $E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \varepsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \varepsilon$
 $F \rightarrow (E) \mid i$

$\text{FIRST}(E) = \{ (, i \}$
 $\text{FIRST}(E') = \{ +, \varepsilon \}$
 $\text{FIRST}(T) = \{ (, i \}$
 $\text{FIRST}(T') = \{ *, \varepsilon \}$
 $\text{FIRST}(F) = \{ (, i \}$

- $i + i$

$\text{FIRST}(TE') = \{ (, i \}$
 $\text{FIRST}(FT') = \{ (, i \}$

$\text{FIRST}(T) = \{ (, i \}$
 $\text{FIRST}(T') = \{ *, \varepsilon \}$
 $\text{FIRST}(TE') = \{ (, i \}$
 $\text{FIRST}(FT') = \{ (, i \}$

i + i #

IP

G(E):

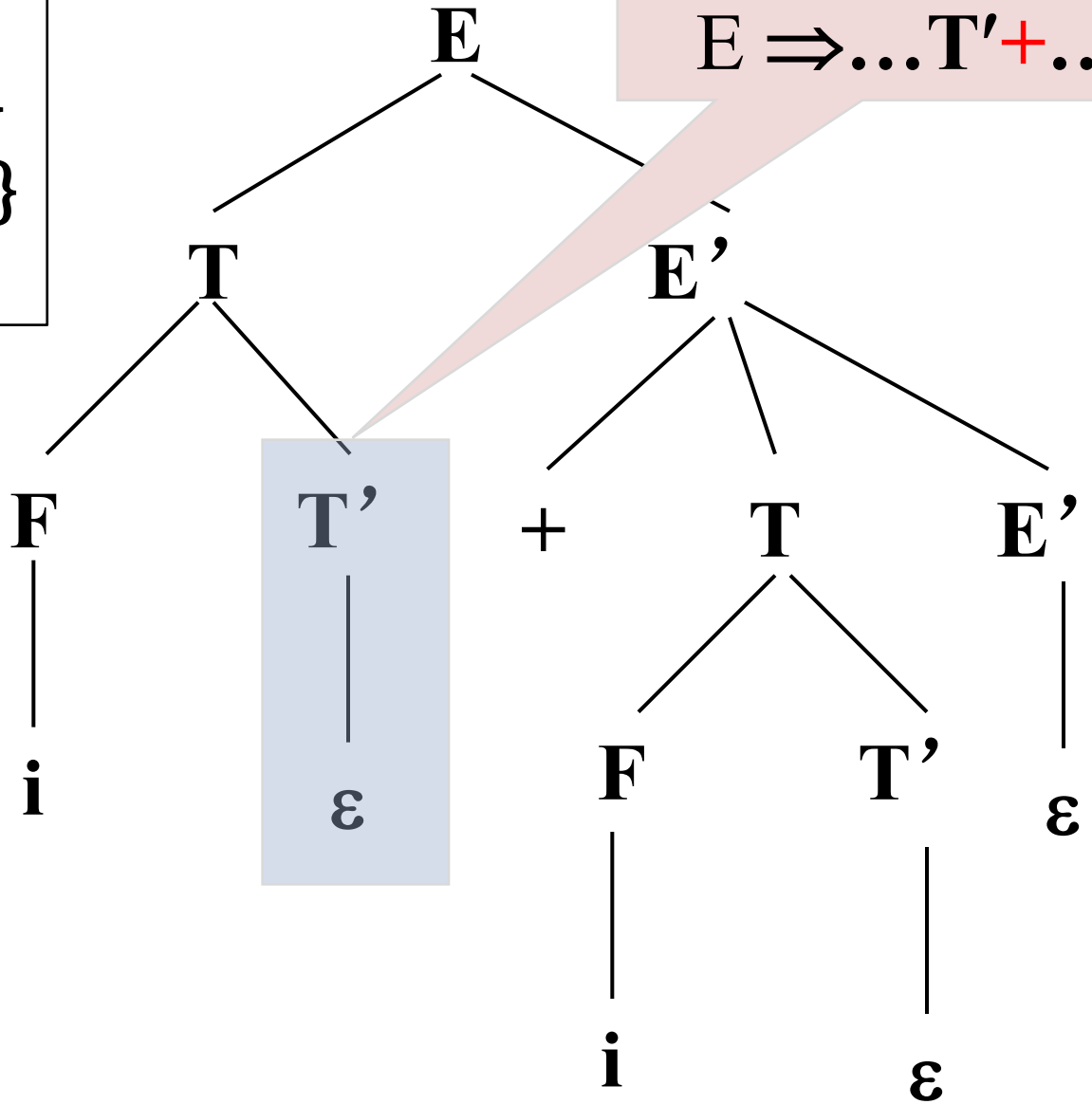
$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$



4.3.3 LL(1)分析条件

是否意味着：

当非终结符A面临输入符号 a ，且 a 不属于A的任意候选首符集，但A的某个候选首符集包含 ϵ 时，就一定可以使A自动匹配？

只有当 a 是文法的某个句型中跟在A后面的终结符时，才可能允许A自动匹配，
否则， a 在这里的出现就是一种语法错误。

4.3.3 LL(1)分析条件

- 假定S是文法G的开始符号，对于G的任何非终结符A，定义

$$FOLLOW(A) = \{a | S \xRightarrow{*} \cdots Aa \cdots, a \in V_T\}$$

- 若 $S \xRightarrow{*} \cdots A$, 则规定 $\# \in FOLLOW(A)$

构造非终结符的FOLLOW集(p78)

对于文法G的每个非终结符A构造FOLLOW(A)

方法:

连续使用下面的规则，直至每个FOLLOW不再增大为止。

构造非终结符的FOLLOW集(p78)

(1) 对于文法的开始符号S，置#于FOLLOW(S)中；

(2) 若 $A \rightarrow \alpha B \beta$ 是一个产生式，则把FIRST(β)- $\{\epsilon\}$ 加至FOLLOW(B)中；

(3) 若 $A \rightarrow \alpha B$ 是一个产生式，或 $A \rightarrow \alpha B \beta$ 是一个产生式而 $\beta \Rightarrow \epsilon$ (即 $\epsilon \in \text{FIRST}(\beta)$)，则把FOLLOW(A)加至FOLLOW(B)中。

例4.7 G(E):

$E \rightarrow TE'$

$E' \rightarrow +T E' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid i$

$\text{FIRST}(E) = \{ (, i \}$

$\text{FIRST}(E') = \{ +, \epsilon \}$

$\text{FIRST}(T) = \{ (, i \}$

$\text{FIRST}(T') = \{ *, \epsilon \}$

$\text{FIRST}(F) = \{ (, i \}$

$\text{FOLLOW}(E) = \{ \#,) \}$

$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \#,) \}$

$\text{FOLLOW}(T) = \text{FIRST}(E') \setminus \{ \epsilon \} \cup \text{FOLLOW}(E)$
 $= \{ + \} \cup \{ \#,) \} = \{ +, \#,) \}$

$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{ +, \#,) \}$

$\text{FOLLOW}(F) = \text{FIRST}(T') \setminus \{ \epsilon \} \cup \text{FOLLOW}(T) \cup$
 $\text{FOLLOW}(T') = \{ *, +, \#,) \}$

■ 构造不带回溯的自上而下分析的文法条件

1. 文法**不含左递归**。

2. 文法中每一个非终结符A的各个产生式的候选首符集两两不相交。即，

若 $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$

则 **$\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \phi$** **$(i \neq j)$**

3. 文法中的每个非终结符A，若它存在某个候选首符集包含 ϵ ，则

$$\text{FIRST}(A) \cap \text{FOLLOW}(A) = \phi$$

如果一个文法G满足以上条件，则称该文法G为**LL(1)文法**。

4.3.3 LL(1)分析条件

- LL(1)
 - L: 从左到右扫描输入串
 - L: 最左推导
 - 1: 每步向前查看1个符号

- 对于一个LL(1)文法，可以对其输入串进行有效的无回溯的自上而下分析。
- 假设要用非终结符A进行匹配，面临的输入符号为a，A的所有产生式为

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$$

1. 若 $a \in \text{FIRST}(\alpha_i)$ ，则指派 α_i 执行匹配任务；
2. 若a不属于任何一个候选首符集，则：
 - (1) 若 ϵ 属于某个 $\text{FIRST}(\alpha_i)$ 且 $a \in \text{FOLLOW}(A)$ ，则让A与 ϵ 自动匹配。
 - (2) 否则，a的出现是一种语法错误。

根据LL(1)文法的条件，每一步工作都是确信无疑的。

4.4 递归下降分析程序构造

- 当文法满足LL(1)条件时，可以为它构造一个不带回溯的自上而下分析程序。

- **递归下降分析程序**：由一组**递归过程**组成，每个过程对应文法的一个**非终结符**。
- 几个全局过程和变量：
 - ADVANCE(), 输入串指示器IP指向下一个输入符号，即读入一个单词符号
 - SYM, IP当前所指的输入符号
 - ERROR(), 出错处理程序

- 例: 文法 $G(E)$:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$

- 每个非终结符有对应的子程序的定义，首先在分析过程中，当需要从某个非终结符出发进行展开(推导)时，就调用这个非终结符对应的子程序。

```
void E();  
void E'();  
void T();  
void T'();  
void F();
```

```
void E(
)
{ T();
  E'();
}
```

```
void T(
)
{ F();
  T'();
}
```

```
void E'()
{ if(sym=='+')
    { advance();
      T();
      E'();
    }
}
```

```
void T'()
{ if(sym=='*')
    { advance();
      F();
      T'();
    }
}
```

```
void F()
{ if(sym=='(')
    { advance();
      E();
      if(sym==')')
        advance();
      else
        error();
    }
  else if(sym=='i')
    advance();
  else error();
}
```

4.5 预测分析程序

- 预测分析程序
 - 使用一张分析表和一个栈进行控制

4.5 预测分析程序

4.5.1 预测分析程序工作过程

4.5.2 预测分析表的构造[↩](#)

4.5.1 预测分析程序工作过程

- 预测分析器模型
 - 栈STACK存放**文法符号**;分析开始时, 栈底先放一个#, 然后, 放进文法开始符号。
 - 输入串之后也总有一个#, 标志输入串结束。
 - 预测分析表是一个 $M[A, a]$ 的矩阵。
 - 总控程序在任何时候都是按STACK栈顶符号X和当前输入符号a行事。

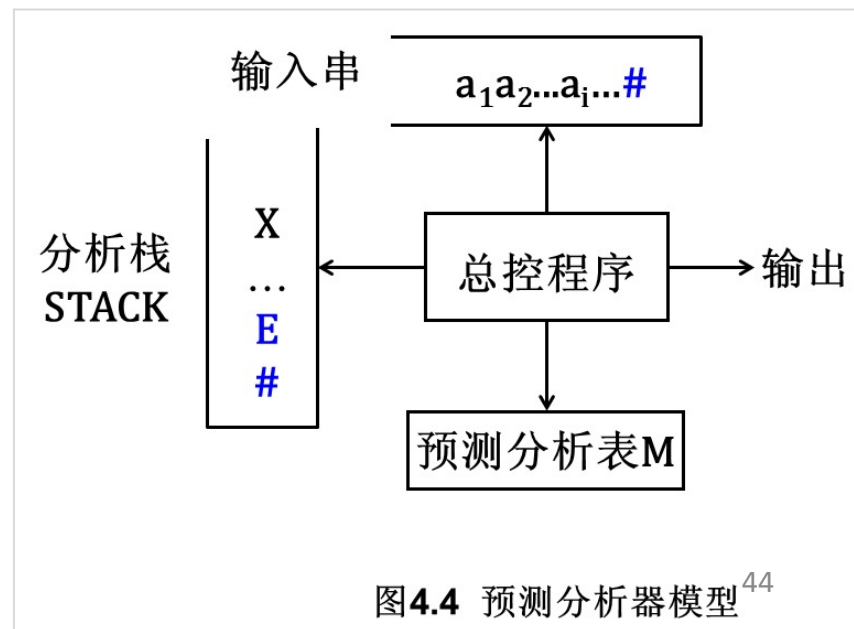


图4.4 预测分析器模型⁴⁴

4.5.1 预测分析程序工作过程

- 对于任何 (X, a) ，总控程序执行下述三种动作之一：
 - (1) 若 $X=a=' \# '$ ，则宣布分析成功，停止分析。
 - (2) 若 $X=a \neq ' \# '$ ，则把 X 从STACK栈顶逐出，让 a 指向下一个输入符号。
 - (3) 若 X 是一个非终结符，则查看分析表 M ：
 - ① 若 $M[A, a]$ 中存放着关于 X 的产生式，把 X 逐出STACK栈顶，把产生式的右部符号串按反序一一推进STACK栈(若右部符号为 ϵ ，则意味不推什么东西进栈)。
 - ② 若 $M[A, a]$ 中存放着“出错标志”，则调用出错诊察程序**ERROR**。

4.5.1 预测分析程序工作过程

- 总控程序

BEGIN

首先把‘#’，然后把文法开始符号推进STACK栈；

把第一个输入符号读进a；

FLAG := TRUE ;

WHILE FLAG DO

BEGIN

栈顶符号上托出去并放在X中；

IF $X \in V_T$ THEN

IF $X=a$ THEN 把下一输入符号读进a /*匹配*/

ELSE ERROR

ELSE IF $X= \text{'#'} \text{' THEN$

IF $X=a$ THEN FLAG:=FALSE /*分析成功*/

ELSE ERROR

ELSE IF $M[A, a] = \{ X \rightarrow X_1 X_2 \cdots X_k \}$ THEN /*推导*/

把 $X_k, X_{k-1}, \dots, X_1$ 一一推进STACK栈

/* 若 $X_1 X_2 \cdots X_k = \varepsilon$ ，不推什么进栈 */

ELSE ERROR

END OF WHILE;

STOP /*分析成功，过程完毕*/

END

4.5.1 预测分析程序工作过程

- 例4.6 文法4.2 $G(E)$:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$

输入串:

$i_1 * i_2 + i_3$

表4.1 文法(4.2)的LL(1)分析表

	i	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

<u>步骤</u>	<u>符号栈</u>	<u>输入串</u>	<u>所用产生式</u>
0	#E	$i_1*i_2+i_3\#$	
1	#E'T	$i_1*i_2+i_3\#$	$E \rightarrow TE'$
2	#E'T'F	$i_1*i_2+i_3\#$	$T \rightarrow FT'$
3	#E'T'i	$i_1*i_2+i_3\#$	$F \rightarrow i$
4	#E'T'	$*i_2+i_3\#$	
5	#E'T'F*	$*i_2+i_3\#$	$T' \rightarrow *FT'$
6	#E'T'F	$i_2+i_3\#$	
7	#E'T'i	$i_2+i_3\#$	$F \rightarrow i$
8	#E'T'	$+i_3\#$	
9	#E'	$+i_3\#$	$T' \rightarrow \varepsilon$
10	#E'T+	$+i_3\#$	$E' \rightarrow +TE'$
11	#E'T	$i_3\#$	
12	#E'T'F	$i_3\#$	$T \rightarrow FT'$
13	#E'T'i	$i_3\#$	$F \rightarrow i$
14	#E'T'	#	
15	#E'	#	$T' \rightarrow \varepsilon$
16	#	#	$E' \rightarrow \varepsilon$

4.5.2 预测分析表的构造

- 预测分析表
 - 矩阵 $M[A, a]$
 - A : 非终结符
 - a : 终结符或 $\#$
 - 元素 $M[A, a]$
 - 关于 A 的产生式, 指出当 A 面临输入符号 a 时所应采用的候选
 - 出错标志, 指出 A 根本不该面临输入符号 a

表4.1 文法(4.2)的LL(1)分析表

	i	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

4.5.2 预测分析表的构造

- 构造算法

- (0) 对文法 G 的每个非终结符 A 及其候选 α 构造 $\text{FIRST}(\alpha)$ 和 $\text{FOLLOW}(A)$;
- (1) 对文法 G 的每个产生式 $A \rightarrow \alpha$ 执行第2步和第3步;
- (2) 对每个终结符 $a \in \text{FIRST}(\alpha)$, 把 $A \rightarrow \alpha$ 加至 $M[A, a]$ 中 ;
- (3) 若 $\epsilon \in \text{FIRST}(\alpha)$, 则对任何 $b \in \text{FOLLOW}(A)$, 把 $A \rightarrow \alpha$ 加至 $M[A, b]$ 中;
- (4) 把所有无定义的 $M[A, a]$ 标上“出错标志”。

例4.7 $G(E)$:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$

$\text{FIRST}(E) = \{ (, i \}$

$\text{FIRST}(E') = \{ +, \varepsilon \}$

$\text{FIRST}(T) = \{ (, i \}$

$\text{FIRST}(T') = \{ *, \varepsilon \}$

$\text{FIRST}(F) = \{ (, i \}$

$\text{FOLLOW}(E) = \{ \#,) \}$

$\text{FOLLOW}(E') = \{ \#,) \}$

$\text{FOLLOW}(T) = \{ +, \#,) \}$

$\text{FOLLOW}(T') = \{ +, \#,) \}$

$\text{FOLLOW}(F) = \{ *, +, \#,) \}$

$\text{FIRST}(TE') = \text{FIRST}(T) = \{ (, i \}$

$\text{FIRST}(+TE') = \{ + \}$

$\text{FIRST}(FT') = \text{FIRST}(F) = \{ (, i \}$

$\text{FIRST}(*FT') = \{ * \}$

$\text{FIRST}((E)) = \{ (\}$

$\text{FIRST}(i) = \{ i \}$

例4.7 $G(E)$: $E \rightarrow TE'$ $E' \rightarrow +T E' \mid \varepsilon$ $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \varepsilon$ $F \rightarrow (E) \mid i$	$FIRST(TE') = \{ (, i \}$ $FIRST(+TE') = \{ + \}$ $FIRST(FT') = \{ (, i \}$ $FIRST(*FT') = \{ * \}$ $FIRST((E)) = \{ (\}$ $FIRST(i) = \{ i \}$	$FOLLOW(E) = \{ \#,) \}$ $FOLLOW(E') = \{ \#,) \}$ $FOLLOW(T) = \{ +, \#,) \}$ $FOLLOW(T') = \{ +, \#,) \}$ $FOLLOW(F) = \{ *, +, \#,) \}$
--	---	---

表4.1 文法(4.2)的LL(1)分析表

	i	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +T E'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

4.6 LL(1)分析中的错误处理

- 预测分析中，遇到语法错误：
 - 栈顶的终结符与当前输入的符号不匹配；
 - 非终结符A处于栈顶，面临的输入符号为a，但分析表M中的M[A, a]为空。
- 恢复策略
 - 跳过输入串中的一些符号直至遇到“同步符号”为止。
- 同步符号集的选择
 - 把FOLLOW(A)中的所有符号放入非终结符A的同步符号集。

如果跳读一些输入符号直到出现FOLLOW(A)中的符号，把A从栈中弹出，可能使分析继续下去。

4.6 LL(1)分析中的错误处理

- 例： **synch** 表示由相应非终结符的FOLLOW集得到的同步符号，对表4.1加入**synch**后见表4.2。

例4.7 $G(E)$:

$E \rightarrow TE'$

$E' \rightarrow +T E' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$

$FOLLOW(E) = \{ \#,) \}$

$FOLLOW(E') = \{ \#,) \}$

$FOLLOW(T) = \{ +, \#,) \}$

$FOLLOW(T') = \{ +, \#,) \}$

$FOLLOW(F) = \{ *, +, \#,) \}$

表4.2 加入同步符号的LL(1)分析表

	i	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$	synch	synch
E'		$E' \rightarrow +T E'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$	synch		$T \rightarrow FT'$	synch	synch
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow i$	synch	synch	$F \rightarrow (E)$	synch	synch

4.6 LL(1)分析中的错误处理

分析时

- 若 $M[A, a]$ 为空，则跳过输入符号 a ；
- 若该项为同步，则弹出栈顶的非终结符；
- 若栈顶的终结符号不匹配输入符号，则弹出栈顶的终结符。

例4.9 带有错误恢复的语法分析过程，输入串为 $+i*+i$

栈	输入	备注
#E	$+i*+i\#$	错误，跳过+
#E	$i*+i\#$	
#E'T	$i*+i\#$	
#E'T'F	$i*+i\#$	
#E'T'i	$i*+i\#$	
#E'T'	$*+i\#$	
#E'T'F*	$*+i\#$	错误, $M[F,+] = \text{synch}$ F 已被弹出
#E'T'F	$+i\#$	
#E'T'	$+i\#$	
#E'	$+i\#$	
#E'T+	$+i\#$	
#E'T	$i\#$	
#E'T'F	$i\#$	
#E'T'i	$i\#$	
#E'T'	$\#$	
#E'	$\#$	
#	$\#$	